

Sistema de Gestión Veterinaria
Validación funcional de API REST mediante Swagger

Documentación de API

Laura Sofia Cangrejo
María Ferdanda Robayo Laguna
Andres Chambo Gonzalez

Arquitectura de Software

Luis Ángel Vargas Narvaez

Ingeniería De Sistema

Universidad CorHuila

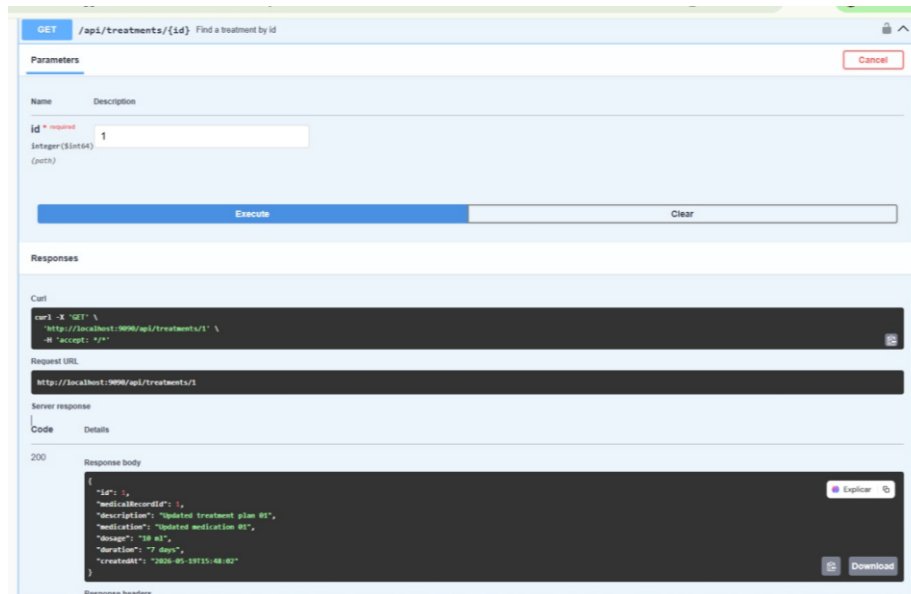
Fecha:
20/05/2026

INTRODUCION

El presente documento expone la validación funcional del backend del Sistema de Gestión Veterinaria mediante pruebas realizadas en Swagger. Estas pruebas permiten comprobar el correcto funcionamiento de los principales módulos del sistema, validando operaciones de creación, consulta, actualización, listado y eliminación de registros. El objetivo es demostrar que la API REST responde de manera coherente, estructurada y alineada con los procesos operativos de una clínica veterinaria.

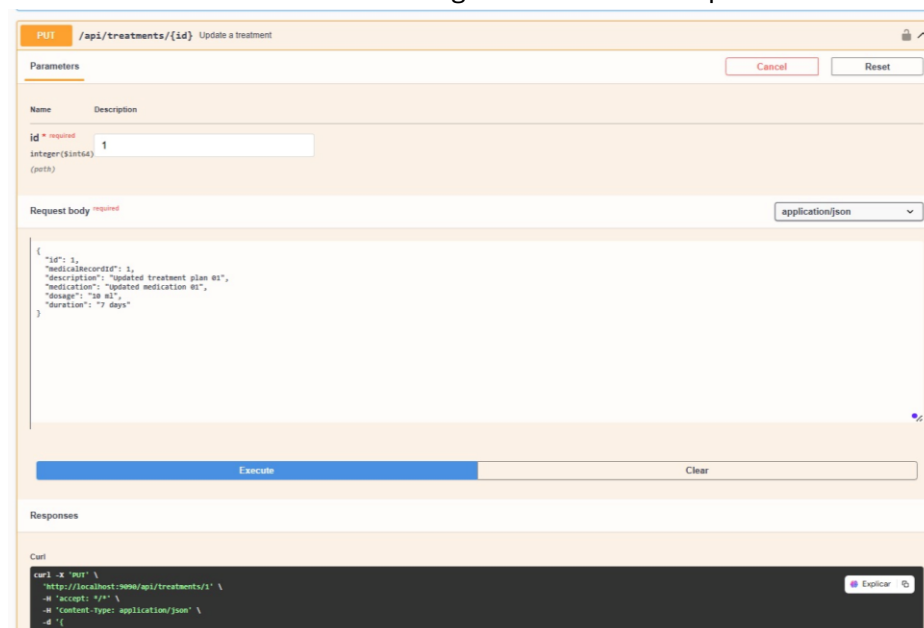
1. Consulta individual de tratamiento

En esta prueba se valida la capacidad del sistema para consultar un tratamiento médico específico mediante su identificador. La API responde correctamente con la información clínica registrada, incluyendo descripción, medicamento, dosis, duración y fecha de creación. Esta validación demuestra que el sistema puede recuperar información médica puntual de forma precisa, estructurada y confiable.



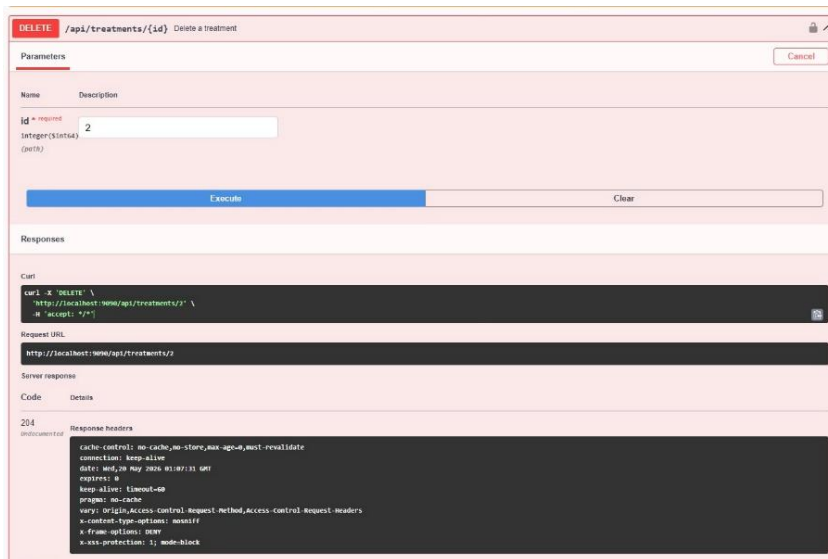
2. Actualización de tratamiento

Esta prueba verifica el proceso de modificación de un tratamiento previamente registrado. El sistema recibe nuevos datos clínicos en formato JSON y actualiza la información asociada al tratamiento correspondiente. Esta funcionalidad es importante porque permite mantener la información médica actualizada según la evolución del paciente.



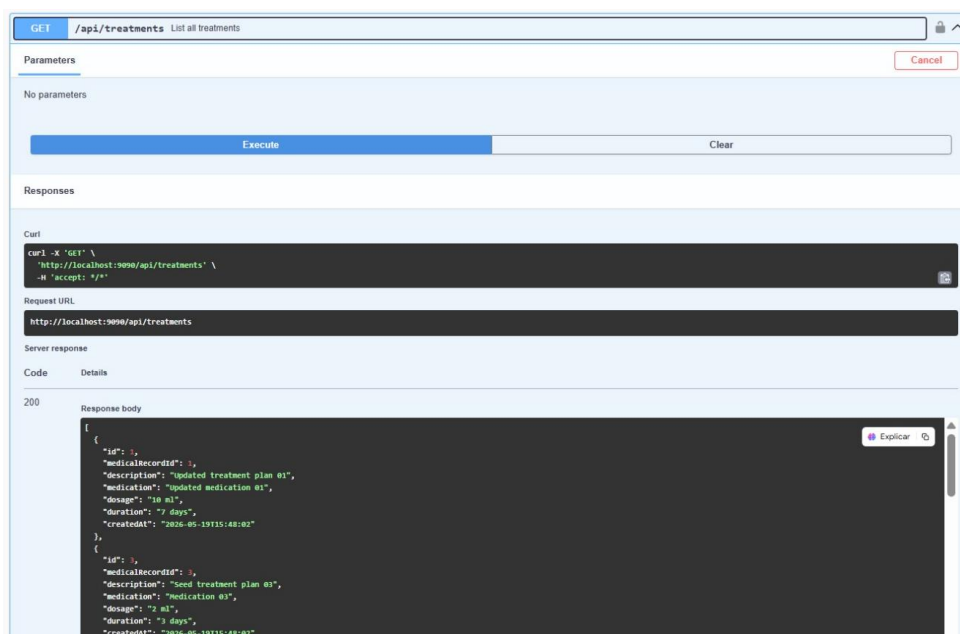
3. Eliminación de tratamiento

En esta prueba se valida la eliminación controlada de un tratamiento dentro del sistema. Al enviar el identificador correspondiente, la API procesa la solicitud y responde satisfactoriamente. Esto demuestra que el backend permite administrar el ciclo de vida de los tratamientos, eliminando registros cuando dejan de ser necesarios o cuando deben corregirse procesos internos.



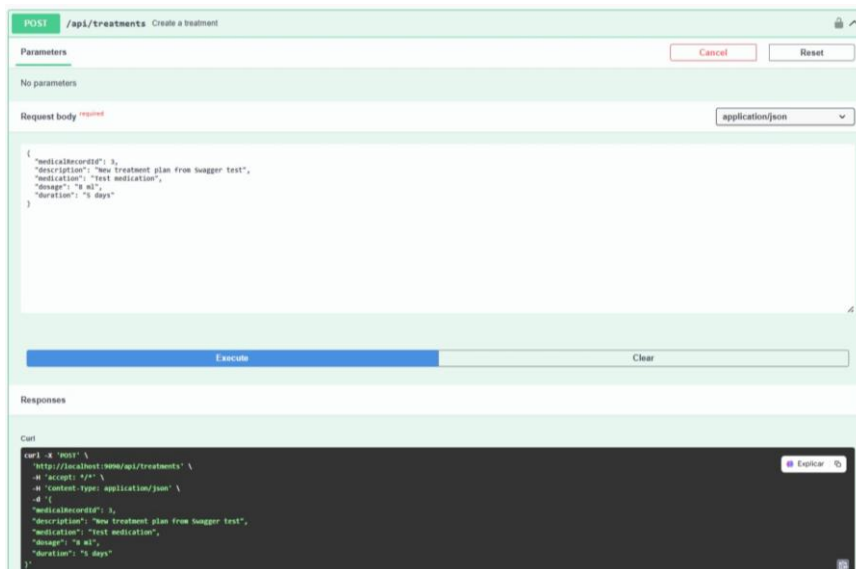
4. Listado general de tratamientos

Esta prueba permite comprobar que el sistema puede consultar y presentar todos los tratamientos registrados. La API devuelve una lista estructurada con los datos principales de cada tratamiento. Desde una perspectiva funcional, esta operación permite tener una visión general de los tratamientos médicos asociados a los historiales clínicos.



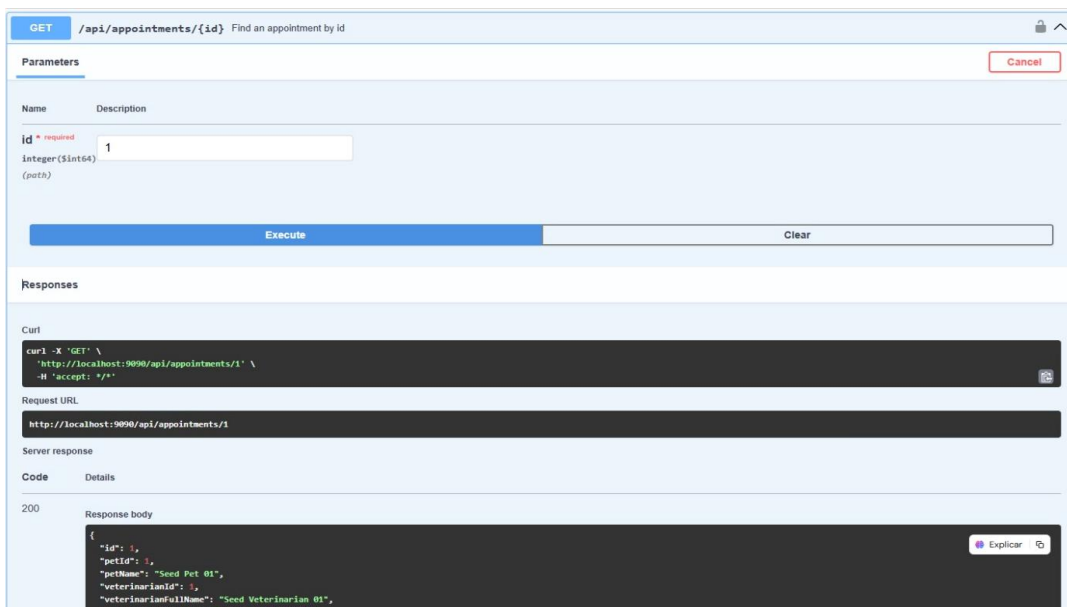
5. Registro de nuevo tratamiento

En esta prueba se valida la creación de un nuevo tratamiento médico dentro del sistema. Se envían datos como historial médico asociado, descripción, medicamento, dosis y duración. La respuesta del servidor confirma que el backend puede registrar información clínica de manera organizada, garantizando trazabilidad en la atención veterinaria.



6. Consulta individual de cita

Esta prueba valida la consulta de una cita específica mediante su identificador. El sistema responde con información relacionada con la mascota, el veterinario, la fecha, la hora, el motivo y el estado de la cita. Esto demuestra que el módulo de citas permite acceder de forma precisa a la programación médica registrada.



7. Actualización de cita

En esta prueba se verifica la modificación de una cita existente. El sistema recibe nuevos datos relacionados con la mascota, el veterinario, la fecha, la hora, el motivo y el estado de la atención. Esta funcionalidad es esencial para permitir reprogramaciones, ajustes administrativos o cambios en el seguimiento clínico.

The screenshot shows a REST client interface with the following details:

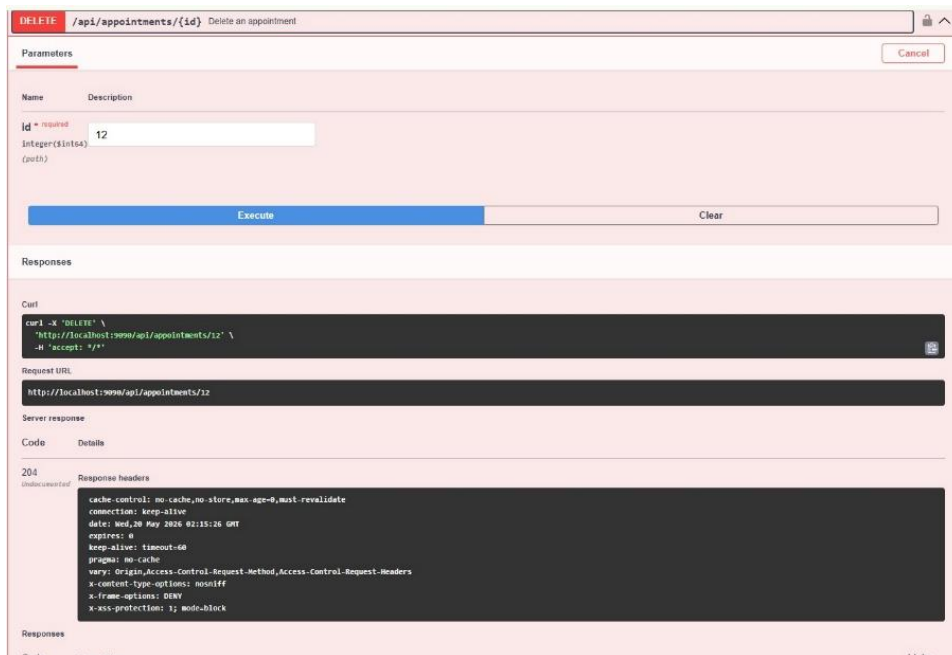
- URL:** `/api/appointments/{id}` with the description "Update an appointment".
- Parameters:** A table with two columns: "Name" and "Description". It contains one parameter: `id` (required), `integer($int64)`, with the value `1` entered in the input field.
- Request body:** Set to `application/json`. The body contains a JSON object:

```
{  "id": 1,  "petId": 1,  "veterinarianId": 1,  "appointmentDateTime": "2025-06-01T09:30:00",  "reason": "Updated follow-up appointment from Snuggles test",  "status": "SCHEDULED"}
```
- Buttons:** "Execute" (highlighted in blue) and "Clear".
- Responses:** A section labeled "Curl" showing the equivalent curl command:

```
curl -X PUT \  -H 'Host: www/api/appointments/1' \  -H 'accept: */*' \  -H 'content-type: application/json' \  -d '{  "id": 1,  "petId": 1,  "veterinarianId": 1,  "appointmentDateTime": "2025-06-01T09:30:00",  "reason": "Updated follow-up appointment from Snuggles test",  "status": "SCHEDULED"  }'
```

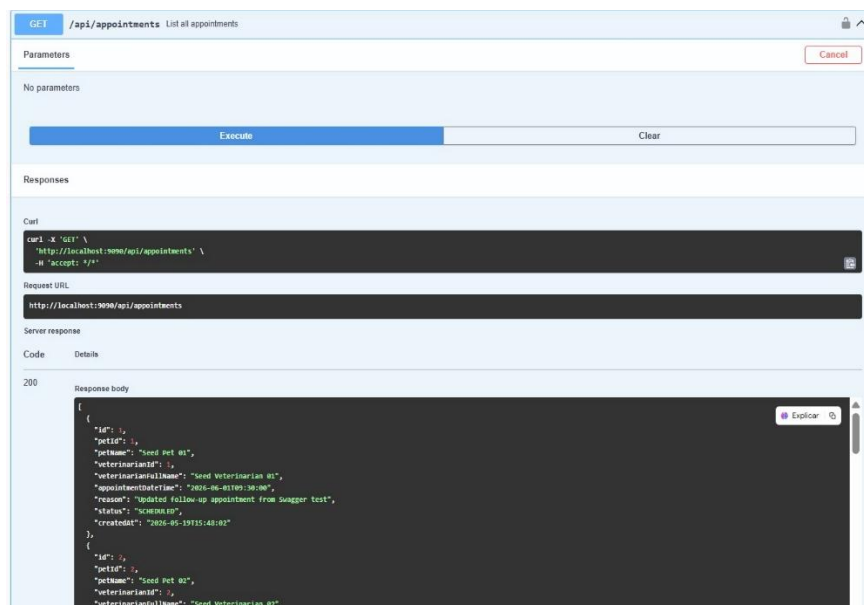
8. Eliminación de cita

Esta prueba valida la eliminación de una cita médica registrada. La API recibe el identificador de la cita y procesa correctamente la solicitud. Esto demuestra que el sistema puede gestionar la cancelación de citas y mantener limpia la información operativa del calendario veterinario.



9. Listado general de citas

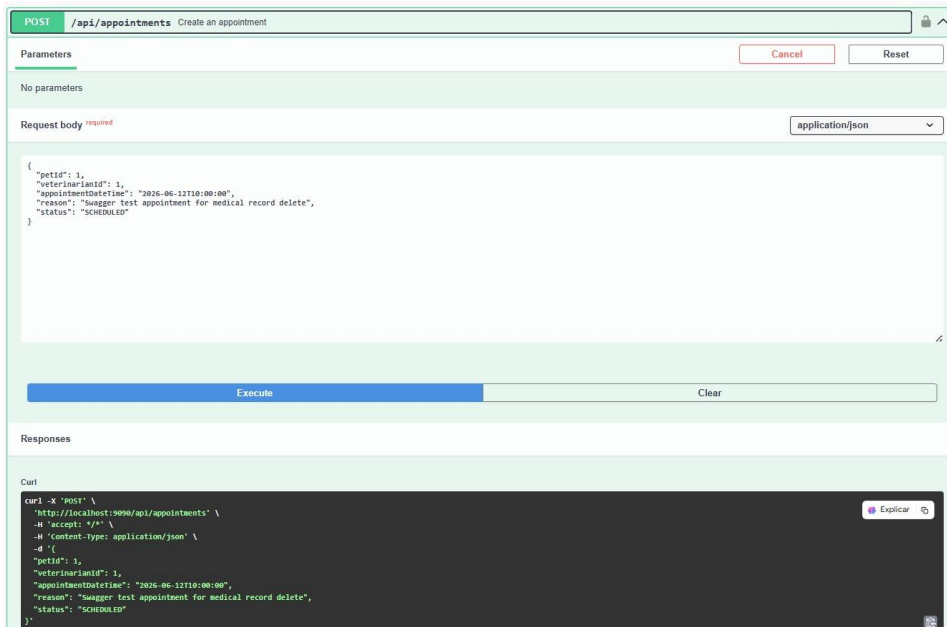
En esta prueba se consulta el listado completo de citas registradas en el sistema. La API devuelve la información organizada de las atenciones programadas. Esta funcionalidad permite al personal administrativo y médico visualizar la agenda general y tener control sobre la operación diaria de la clínica.



10. Registro de nueva cita

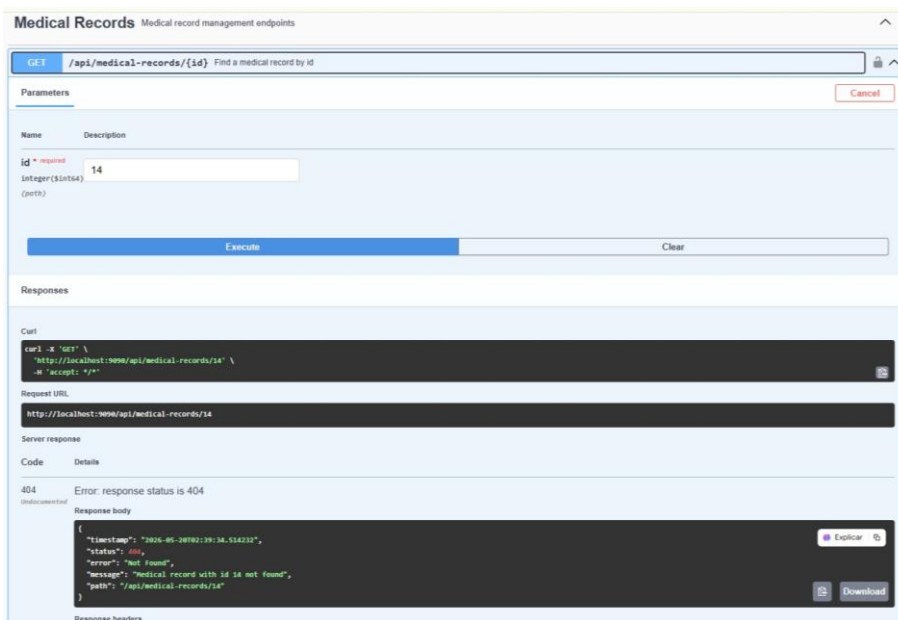
Esta prueba valida la creación de una nueva cita médica. Se envían los datos necesarios para asociar una mascota, un veterinario, una fecha, una hora, un motivo y un estado inicial. La

respuesta exitosa confirma que el sistema permite registrar nuevas atenciones de manera controlada y coherente.



11. Validación de historial médico no encontrado

En esta prueba se consulta un historial médico con un identificador inexistente. El sistema responde correctamente con un mensaje de error controlado, indicando que el registro no fue encontrado. Esta validación es importante porque demuestra que la API no solo gestiona operaciones exitosas, sino también escenarios de excepción de forma adecuada.



12. Actualización de historial médico

Esta prueba verifica la actualización de un historial médico existente. El sistema recibe información clínica modificada, como descripción, diagnóstico y observaciones. Esta funcionalidad permite mantener vigente el expediente médico de cada mascota y soportar un seguimiento clínico más preciso.

The screenshot shows a REST client interface for the **PUT /api/medical-records/{id}** endpoint, titled "Update a medical record".

- Parameters:** A table with columns "Name" and "Description". It contains one parameter: **id** (required), type **integer(int32)**, with a value of **1**.
- Request body:** A text area containing a JSON object:

```
{  "id": 1,  "appointmentId": 1,  "diagnosis": "Updated diagnosis from Swagger test",  "notes": "Updated clinical notes from Swagger test",  "weight": 8.2,  "temperature": 38.4}
```

. The media type is set to **application/json**.
- Buttons:** "Execute" (blue) and "Clear".
- Responses:** A section for viewing the response, currently empty.

13. Eliminación de historial médico

En esta prueba se valida la eliminación de un historial médico específico. El backend procesa la solicitud mediante el identificador enviado y confirma la operación. Esto demuestra que el sistema permite administrar los registros clínicos con control, respetando la estructura lógica de la información médica.

The screenshot shows a REST client interface for the **DELETE /api/medical-records/{id}** endpoint, titled "Delete a medical record".

- Parameters:** A table with columns "Name" and "Description". It contains one parameter: **id** (required), type **integer(int32)**, with a value of **13**.
- Buttons:** "Execute" (blue) and "Clear".
- Responses:** A section showing the response details after execution.

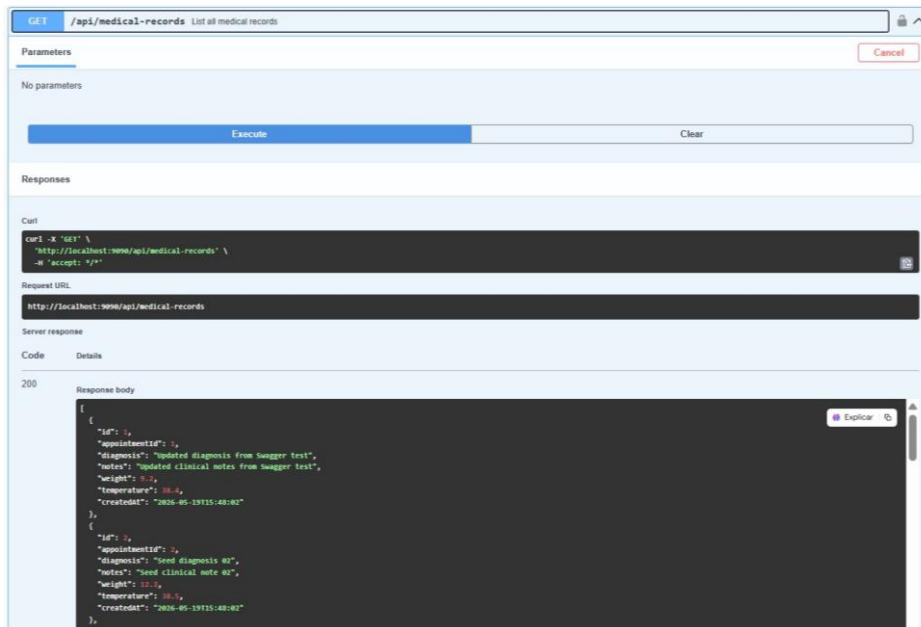
Response details:

- Code:** 204
- Details:** **no-content**
- Response headers:**

```
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Connection: keep-alive
Date: Wed, 09 May 2018 02:47:09 GMT
Expires: 0
Keep-Alive: timeout=60
Pragma: no-cache
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
```

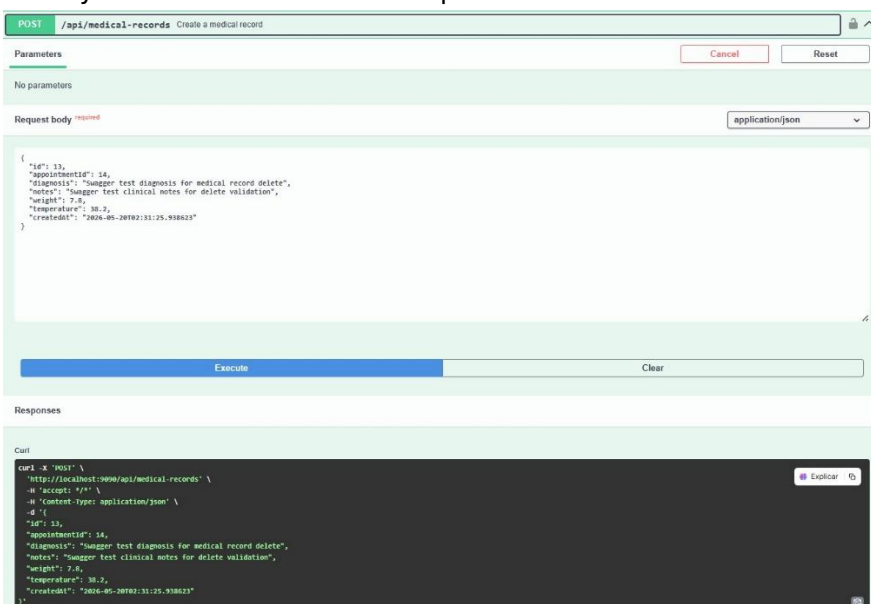
14. Listado general de historiales médicos

Esta prueba comprueba que el sistema puede listar todos los historiales médicos registrados. La API responde con información clínica estructurada, permitiendo visualizar diagnósticos, observaciones y datos asociados a las mascotas. Esta operación fortalece la trazabilidad médica dentro del sistema.



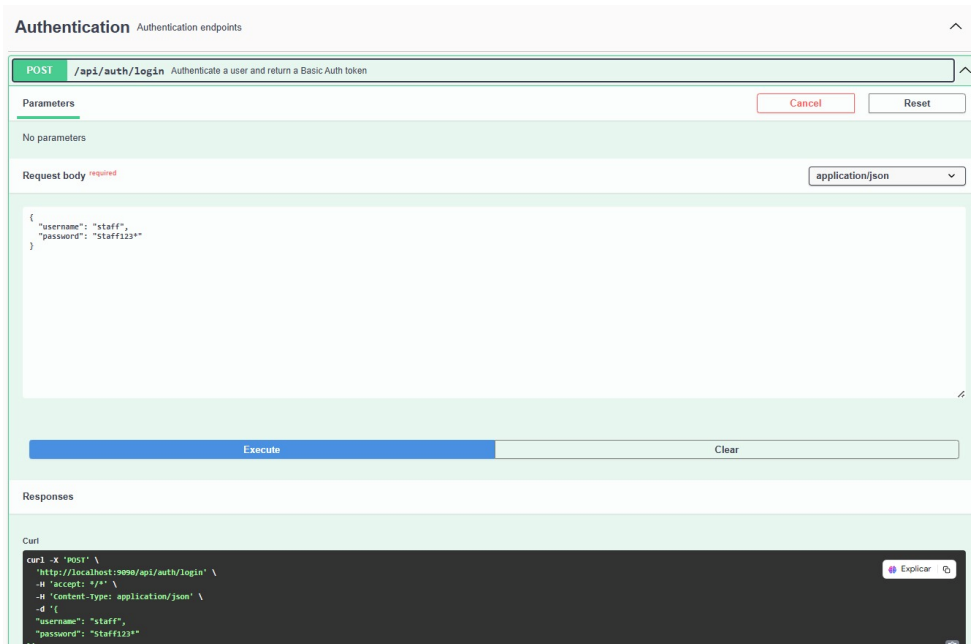
15. Registro de historial médico

En esta prueba se valida la creación de un nuevo historial médico. Se envía información clínica asociada a una mascota, incluyendo descripción, diagnóstico y observaciones. La respuesta exitosa confirma que el sistema permite documentar formalmente el estado de salud y la evolución médica de los pacientes veterinarios.



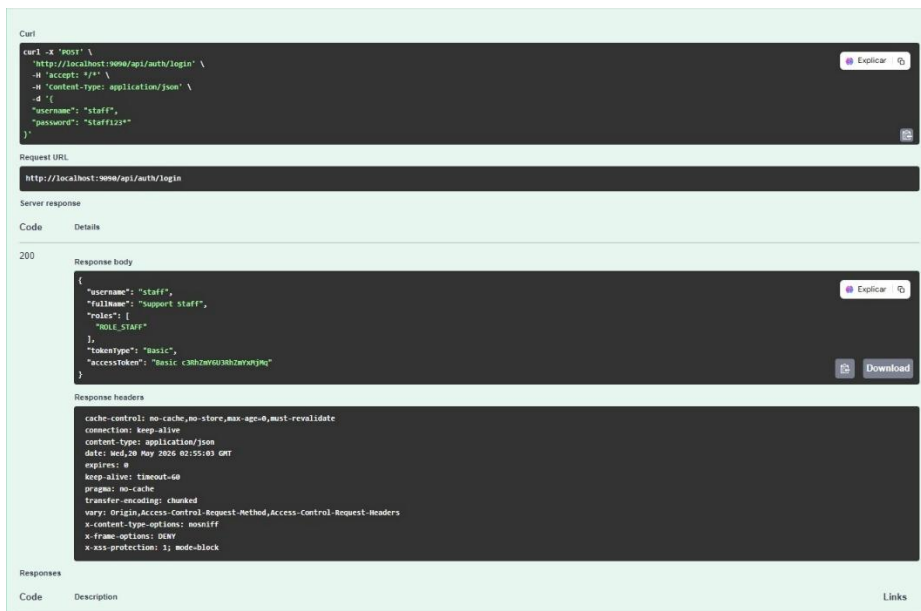
16. Inicio de sesión en el sistema

Esta prueba valida el proceso de autenticación del sistema. Se envían credenciales de acceso al endpoint correspondiente y el backend procesa la solicitud para verificar la identidad del usuario. Esta funcionalidad es fundamental porque protege el acceso a la información y garantiza que solo usuarios autorizados puedan interactuar con el sistema.



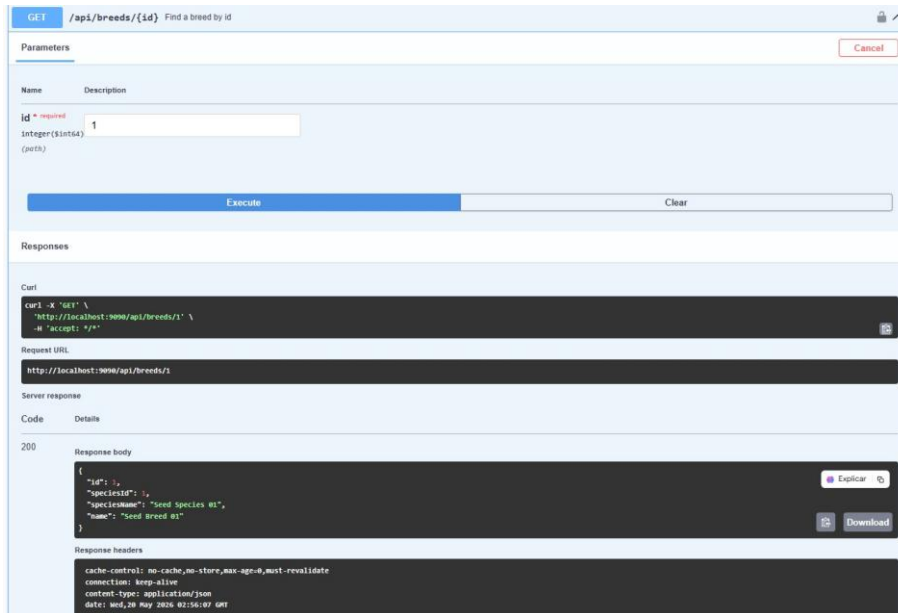
17. Confirmación de autenticación

En esta prueba se evidencia la respuesta generada después del inicio de sesión. El sistema devuelve una respuesta válida, confirmando que las credenciales fueron procesadas correctamente. Esto demuestra que el módulo de autenticación funciona como punto de entrada seguro para las operaciones del backend.



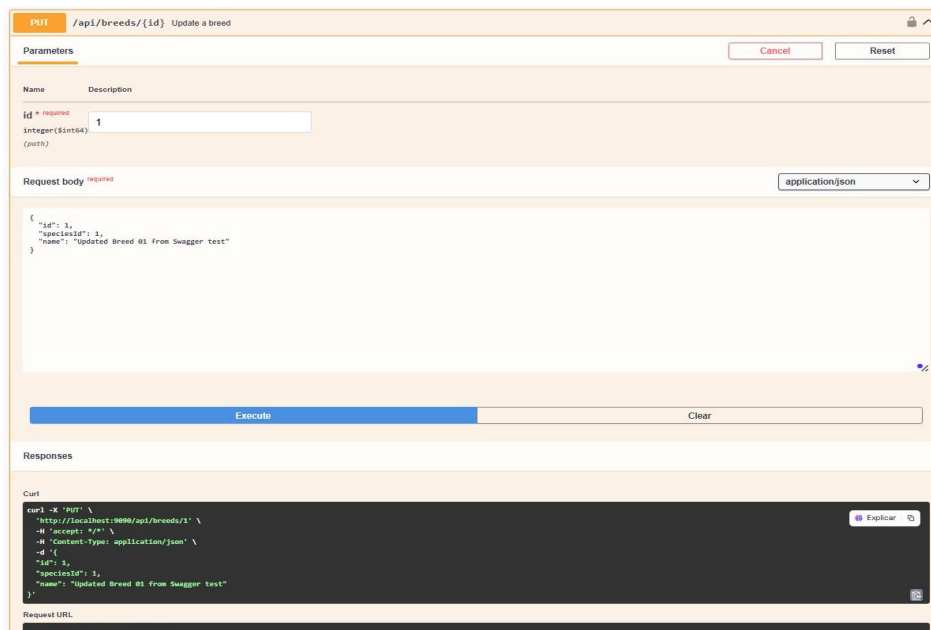
18. Consulta individual de raza

Esta prueba valida la consulta de una raza específica mediante su identificador. El sistema responde con información de la raza y su relación con una especie determinada. Esta operación confirma que el backend mantiene correctamente la relación entre especies y razas, un aspecto importante para clasificar adecuadamente a las mascotas.



19. Actualización de raza

En esta prueba se verifica la actualización de una raza registrada. La API recibe datos modificados, como el nombre de la raza y la especie asociada. Esta funcionalidad permite mantener actualizado el catálogo de razas utilizado por el sistema.



20. Eliminación de raza

Esta prueba valida la eliminación de una raza específica. El backend procesa la solicitud correctamente y confirma que el registro fue eliminado. Esto demuestra que el sistema permite administrar los catálogos de clasificación de manera flexible y controlada.

DELETE /api/breeds/{id} Delete a breed

Parameters

Name	Description
id ^{required}	
integer(\$int64)	
(path)	

Execute Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:9090/api/breeds/11' \
  -H 'accept: */*'
```

Request URL

http://localhost:9090/api/breeds/11

Server response

Code Details

204

Response headers

```
cache-control: no-cache, no-store, max-age=0, must-revalidate
connection: keep-alive
date: Wed, 29 May 2024 04:51:29 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
vary: Origin, Access-Control-Request-Method, Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-ssr-protection: 1; mode=block
```

Responses

Code	Description	Links
200	OK	No links

21. Listado general de razas

En esta prueba se consulta el listado completo de razas registradas. El sistema devuelve una colección estructurada de datos, incluyendo la relación con las especies correspondientes. Esta funcionalidad permite visualizar y administrar el catálogo de razas disponible para el registro de mascotas.

GET /api/breeds List all breeds

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:9090/api/breeds' \
  -H 'accept: */*'
```

Request URL

http://localhost:9090/api/breeds

Server response

Code Details

200

Response body

```
{
  "id": 1,
  "speciesId": 1,
  "speciesName": "Seed Species 01",
  "name": "Updated Breed 01 from Swagger test"
},
{
  "id": 2,
  "speciesId": 2,
  "speciesName": "Seed Species 02",
  "name": "Seed Breed 02"
},
{
  "id": 3,
  "speciesId": 3,
  "speciesName": "Seed Species 03",
  "name": "Seed Breed 03"
},
{
  "id": 4,
  "speciesId": 4,
  "speciesName": "Seed Species 04",
  "name": "Seed Breed 04"
}
```

Response headers

22. Registro de nueva raza

Esta prueba valida la creación de una nueva raza dentro del sistema. Se envía el nombre de la raza junto con la especie relacionada. La respuesta exitosa confirma que el backend permite ampliar el catálogo de razas de forma organizada.

The image shows a Swagger UI interface for a POST endpoint. The top bar indicates the method is POST and the endpoint is /api/breeds, with a description 'Create a breed'. Below this, the 'Parameters' section is empty, showing 'No parameters'. The 'Request body' section is marked as 'required' and has a dropdown menu set to 'application/json'. The request body is a JSON object: { "speciesId": 1, "name": "Swagger Test Breed" }. Below the request body, there are 'Execute' and 'Clear' buttons. The 'Responses' section is currently empty. At the bottom, there is a 'Curl' section with a terminal-like box containing the following command: curl -X POST \ "http://localhost:3030/api/breeds" \ -H "accept: */*" \ -H "Content-Type: application/json" \ -d { "speciesId": 1, "name": "Swagger Test Breed" }. Below the curl command, there is a 'Request URL' section showing 'http://localhost:3030/api/breeds'. At the very bottom, there is a 'Server response' section with 'Code' and 'Details' tabs.

POST /api/breeds Create a breed

Parameters

No parameters

Request body *required*

application/json

```
{  "speciesId": 1,  "name": "Swagger Test Breed"}
```

Execute Clear

Responses

Curl

```
curl -X POST \  "http://localhost:3030/api/breeds" \  -H "accept: */*" \  -H "Content-Type: application/json" \  -d {    "speciesId": 1,    "name": "Swagger Test Breed"  }
```

Request URL

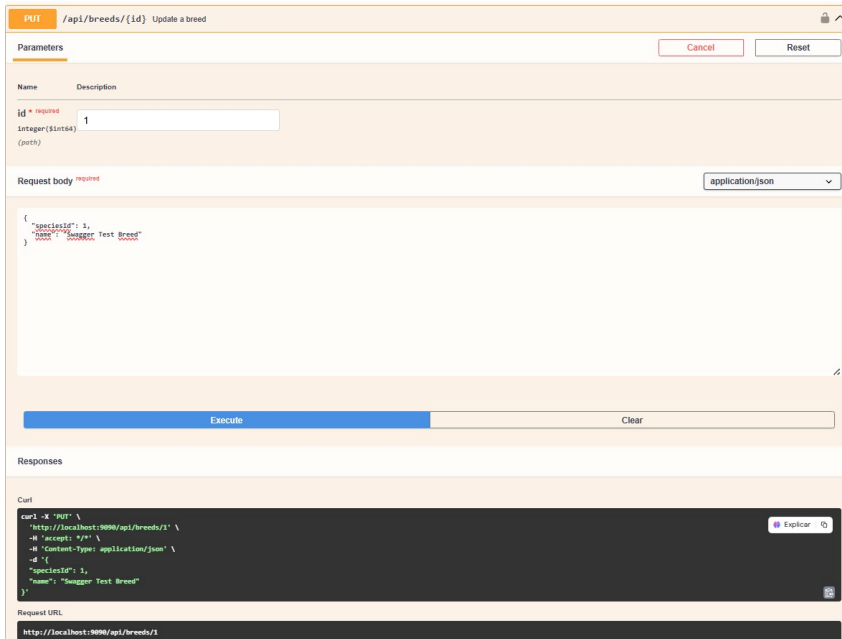
http://localhost:3030/api/breeds

Server response

Code Details

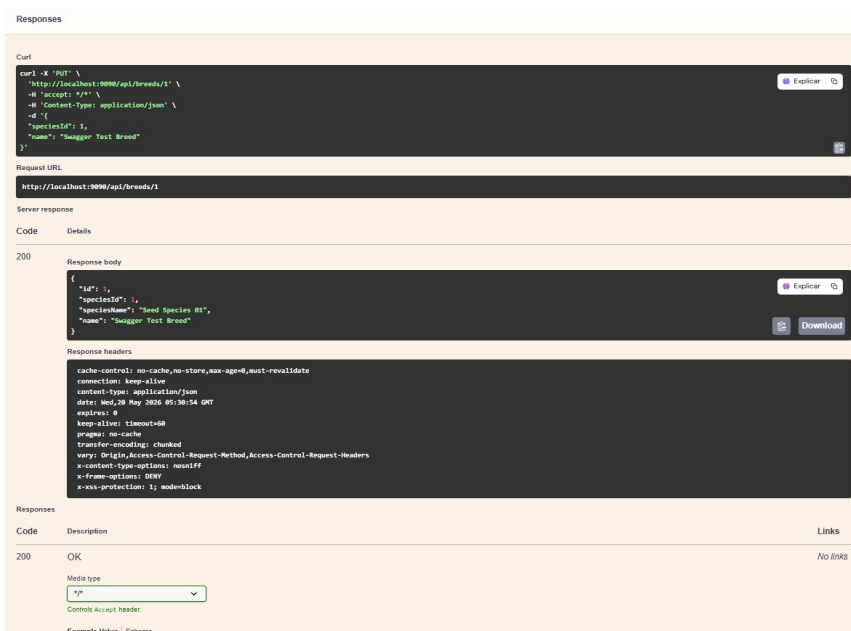
23. Actualización de raza con datos de prueba

En esta prueba se realiza una modificación adicional sobre una raza utilizando datos de prueba desde Swagger. Esto permite verificar que el endpoint de actualización responde correctamente ante nuevas entradas y mantiene la consistencia de la información almacenada.



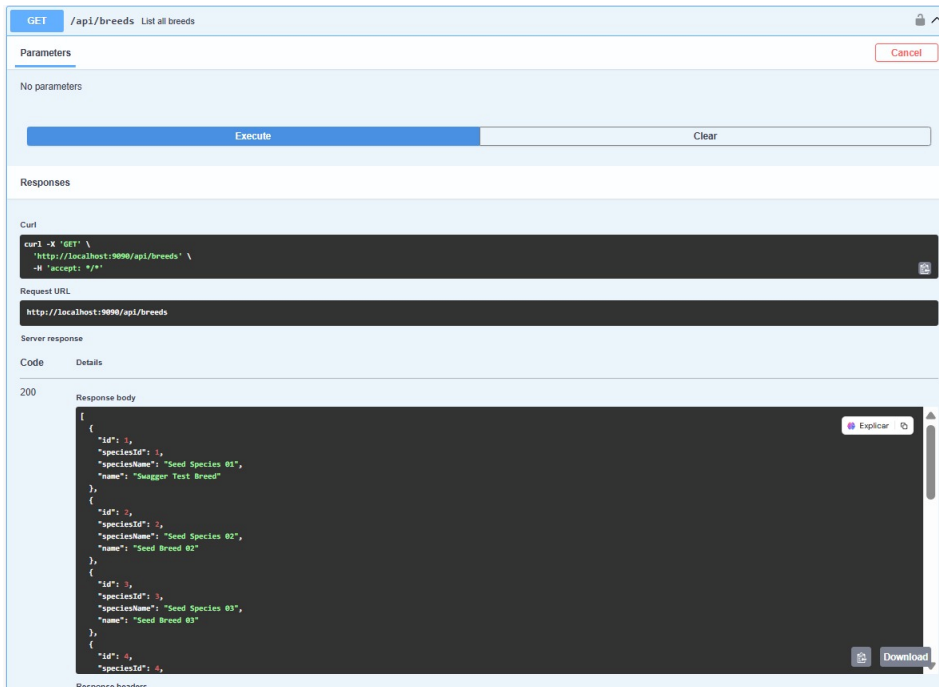
24. Confirmación de actualización de raza

Esta imagen evidencia la respuesta exitosa después de actualizar una raza. El sistema devuelve el registro modificado, confirmando que los cambios fueron aplicados correctamente en la base de datos. Esto valida la confiabilidad del proceso de actualización.



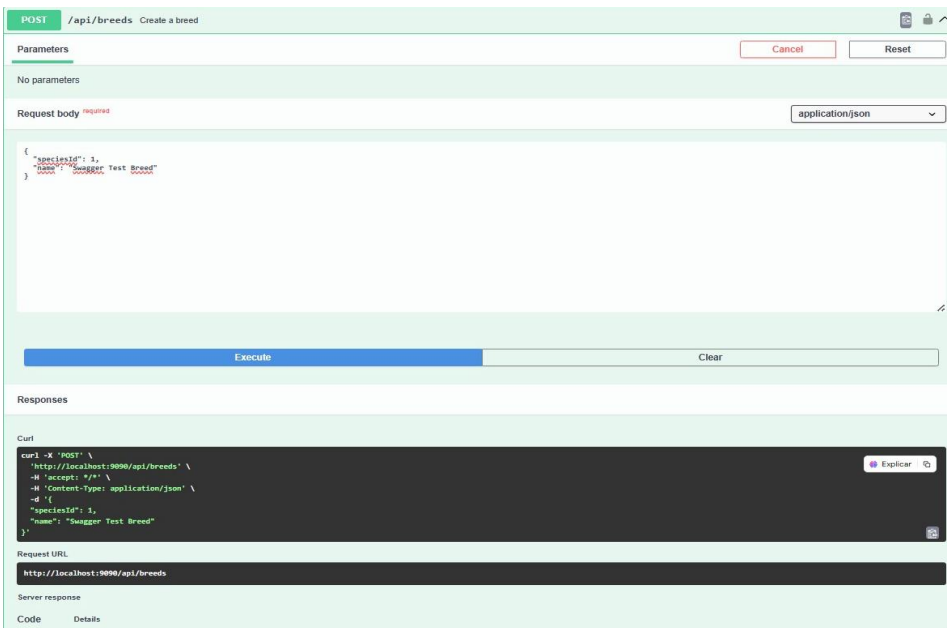
25. Verificación del listado de razas actualizado

En esta prueba se consulta nuevamente el listado de razas para verificar que los cambios realizados previamente aparecen reflejados. Esta validación es importante porque confirma que las operaciones de creación o modificación tienen persistencia real en el sistema.



26. Creación exitosa de raza

Esta prueba evidencia el registro exitoso de una nueva raza. La API responde con un código de creación, indicando que la información fue aceptada y almacenada correctamente. Esto confirma que el módulo de razas cumple con el proceso de alta de nuevos registros.



27. Eliminación de raza creada

En esta prueba se elimina una raza que fue creada durante el proceso de validación. Esto permite comprobar el ciclo completo de gestión del registro: creación, consulta, modificación y eliminación. La respuesta satisfactoria demuestra que el módulo cumple

correctamente con las operaciones CRUD.

DELETE /api/breeds/{id} Delete a breed

Parameters

Name	Description
id * required	integer(int64) (path)

Execute Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:9090/api/breeds/13' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:9090/api/breeds/13
```

Server response

Code Details

204

Response headers

```
cache-control: no-cache, no-store, max-age=0, must-revalidate
connection: keep-alive
date: Wed, 28 May 2026 05:47:05 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
vary: Origin, Access-Control-Request-Method, Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-ssr-protection: 1; mode=block
```

Responses

Code	Description	Links
200	OK	No links

28. Consulta individual de especie

Esta prueba valida la consulta de una especie específica mediante su identificador. El sistema responde con la información correspondiente, demostrando que el módulo de especies permite recuperar registros individuales de forma precisa.

GET /api/species/{id} Find a species by id

Parameters

Name	Description
id * required	integer(int64) (path)

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:9090/api/species/1' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:9090/api/species/1
```

Server response

Code Details

200

Response body

```
{
  "id": 1,
  "name": "Seed Species 01"
}
```

Response headers

```
cache-control: no-cache, no-store, max-age=0, must-revalidate
connection: keep-alive
content-type: application/json
date: Wed, 28 May 2026 05:48:55 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
vary: Origin, Access-Control-Request-Method, Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
```

29. Actualización de especie

En esta prueba se modifica la información de una especie existente. El backend recibe el nuevo nombre de la especie y actualiza el registro correspondiente. Esta funcionalidad permite mantener actualizado el catálogo principal de clasificación animal.

The screenshot shows a REST client interface with the following sections:

- Method and URL:** PUT /api/species/{id} Update a species
- Parameters:** A table with two columns: Name and Description. The first row has 'id' (marked as required) and 'Integer(\$int64) (path)'. The value '1' is entered in the input field.
- Request body:** A text area containing a JSON object:

```
{  "id": 1,  "name": "Swagger Test Species"}

```

 The content type is set to 'application/json'. There are 'Execute' and 'Clear' buttons below the text area.
- Responses:** A section showing the curl command used for the request:

```
curl -X 'PUT' \
  "http://localhost:9090/api/species/1" \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 1,
    "name": "Swagger Test Species"
  }'

```

 Below the curl command is the 'Request URL' field with the value 'http://localhost:9090/api/species/1'.

30. Confirmación de actualización de especie

Esta imagen muestra la respuesta del sistema después de actualizar una especie. La API devuelve el registro modificado, confirmando que la operación fue ejecutada correctamente y

que la información quedó almacenada de forma consistente.

The screenshot displays a REST client interface with the following details:

- Request URL:** `http://localhost:9090/api/species/1`
- Server response:**
 - Code:** 200
 - Response body:**

```
{
  "id": 1,
  "name": "Swagger Test Species"
}
```
 - Response headers:**

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Wed, 20 May 2026 05:52:21 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 1; mode=block
```
- Responses table:**

Code	Description	Links
200	OK	No links
- Media type:** `*/*`
- Controls:** Accept header.
- Example Value / Schema:**

```
{
  "id": 0,
  "name": "string"
}
```

31. Listado general de especies

En esta prueba se consulta el listado completo de especies registradas. El sistema devuelve la información organizada en formato JSON. Esta funcionalidad permite gestionar el catálogo

base sobre el cual se relacionan posteriormente las razas y las mascotas.

GET

/api/species

List all species

Parameters

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:9090/api/species' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:9090/api/species
```

Server response

Code

Details

200

Response body

```
{
  {
    "id": 2,
    "name": "Seed Species 02"
  },
  {
    "id": 3,
    "name": "Seed Species 03"
  },
  {
    "id": 4,
    "name": "Seed Species 04"
  },
  {
    "id": 5,
    "name": "Seed Species 05"
  },
  {
    "id": 6,
    "name": "Seed Species 06"
  }
}
```

Expand

Download

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
```

32. Registro de nueva especie

Esta prueba valida la creación de una nueva especie dentro del sistema. Se envía el nombre correspondiente y el backend procesa el registro. Esta operación permite ampliar la clasificación del sistema de acuerdo con las necesidades de la clínica veterinaria.

POST

/api/species Create a species

Parameters

No parameters

Request body required

application/json

```
{
  "name": "Species Unique Test 20260520"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
'http://localhost:9090/api/species' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
  "name": "Species Unique Test 20260520"
}'
```

Explicar

Request URL

```
http://localhost:9090/api/species
```

Server response

Code

Details

201

Undocumented

Response body

33. Eliminación de especie

En esta prueba se valida la eliminación de una especie registrada. El sistema procesa la solicitud mediante el identificador correspondiente y confirma la eliminación. Esto demuestra que el módulo de especies permite administrar sus registros de manera controlada.

DELETE /api/species/{id} Delete a species

Parameters

Name	Description
id *	required
integer(\$int64)	16
(path)	

Execute Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:9090/api/species/16' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:9090/api/species/16
```

Server response

Code	Details
204	Response headers

Response headers

```
cache-control: no-cache, no-store, max-age=0, must-revalidate
connection: keep-alive
date: Wed, 28 May 2026 06:05:32 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
vary: Origin, Access-Control-Request-Method, Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-ss-protection: 1; mode=block
```

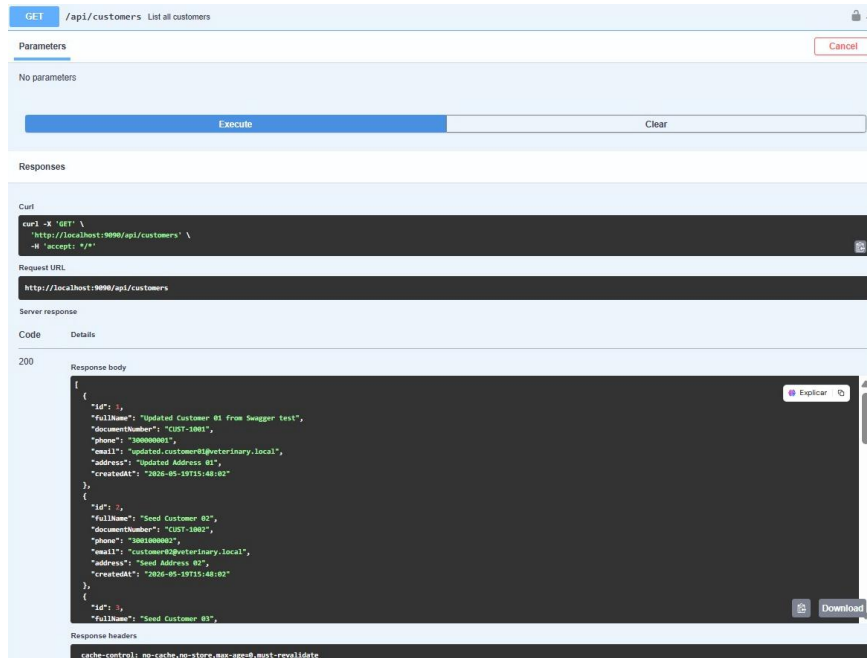
Responses

Code	Description	Links
200	OK	No links

34. Consulta individual de cliente

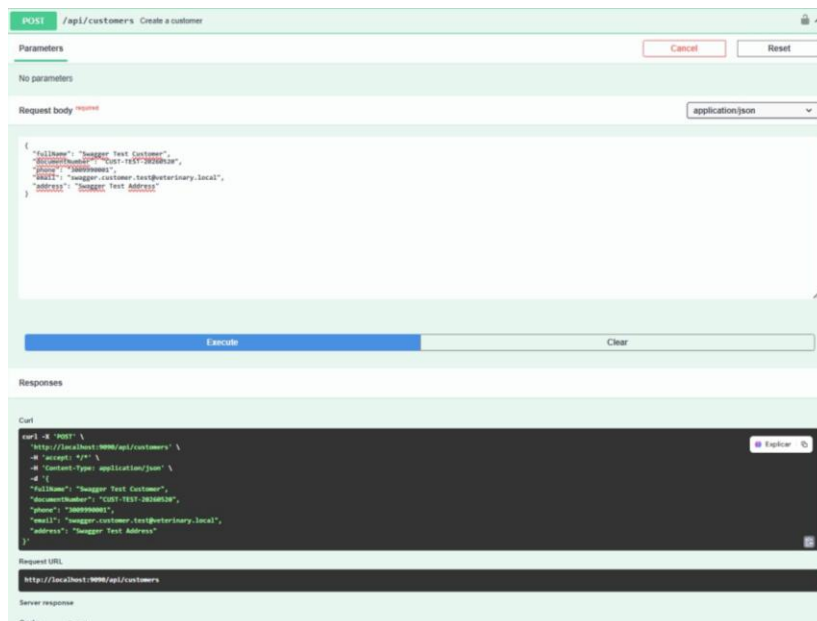
Esta prueba valida la consulta de un cliente específico mediante su identificador. El sistema responde con la información registrada del propietario, como datos personales y de contacto. Esta funcionalidad es clave porque los clientes son la base administrativa para asociar mascotas y servicios veterinarios.

personal administrativo tener control sobre la base de clientes de la clínica.



37. Registro de nuevo cliente

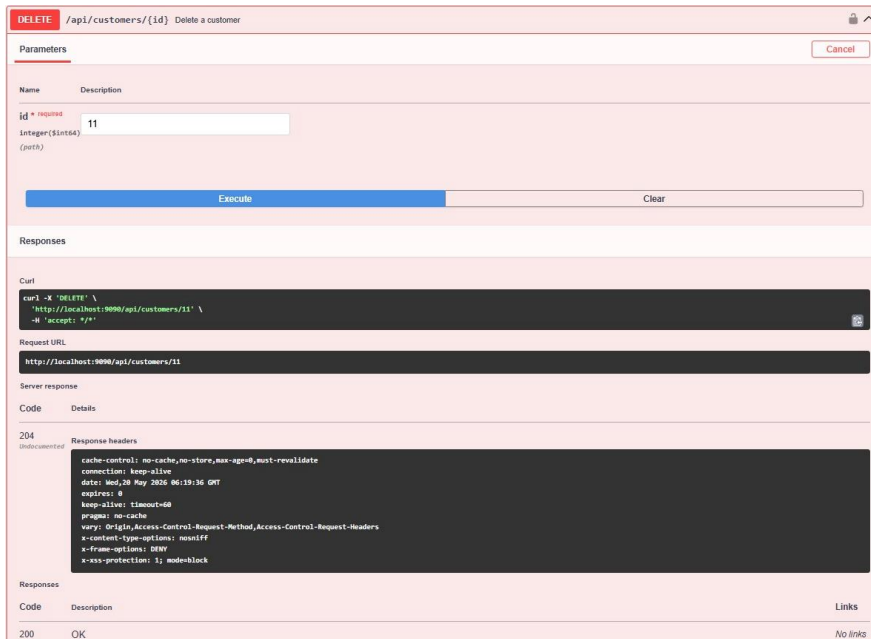
En esta prueba se valida la creación de un nuevo cliente. Se envían los datos necesarios para registrar al propietario dentro del sistema. La respuesta exitosa confirma que el backend puede almacenar nuevos clientes de forma correcta y organizada.



38. Eliminación de cliente

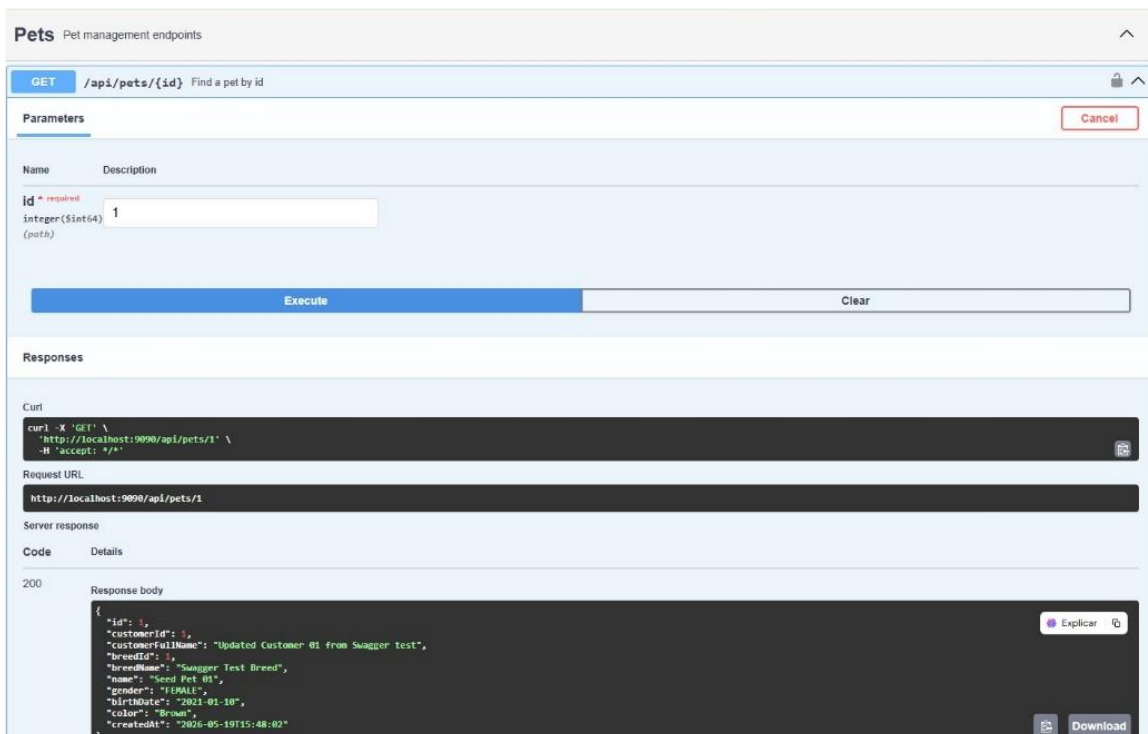
Esta prueba valida la eliminación de un cliente registrado. El sistema recibe el identificador correspondiente y procesa la solicitud de eliminación. Esto demuestra que el módulo de

clientes permite administrar correctamente el ciclo de vida de la información de los propietarios.



39. Consulta individual de mascota

En esta prueba se valida la consulta de una mascota específica mediante su identificador. El sistema devuelve información completa como cliente asociado, raza, nombre, género, fecha de nacimiento, color y fecha de creación. Esta operación confirma que el backend puede recuperar de forma precisa la información de los pacientes veterinarios.



40. Actualización de mascota

Esta prueba verifica la modificación de una mascota existente. El sistema recibe nuevos datos relacionados con su identificación, características físicas y datos generales. Esta funcionalidad permite mantener actualizado el perfil de cada mascota dentro del sistema.

Swagger UI interface for the `POST /api/pets/{id}` endpoint, titled "Update a pet".

Parameters:

Name	Description
<code>id</code> * required	
integer(int32)	
(path)	

Request body: required (Type: `application/json`)

```
{
  "id": 1,
  "customerId": 1,
  "breedId": 1,
  "name": "Updated Pet #1 from Swagger test",
  "gender": "FEMALE",
  "birthDate": "2021-01-10",
  "color": "Updated Brown"
}
```

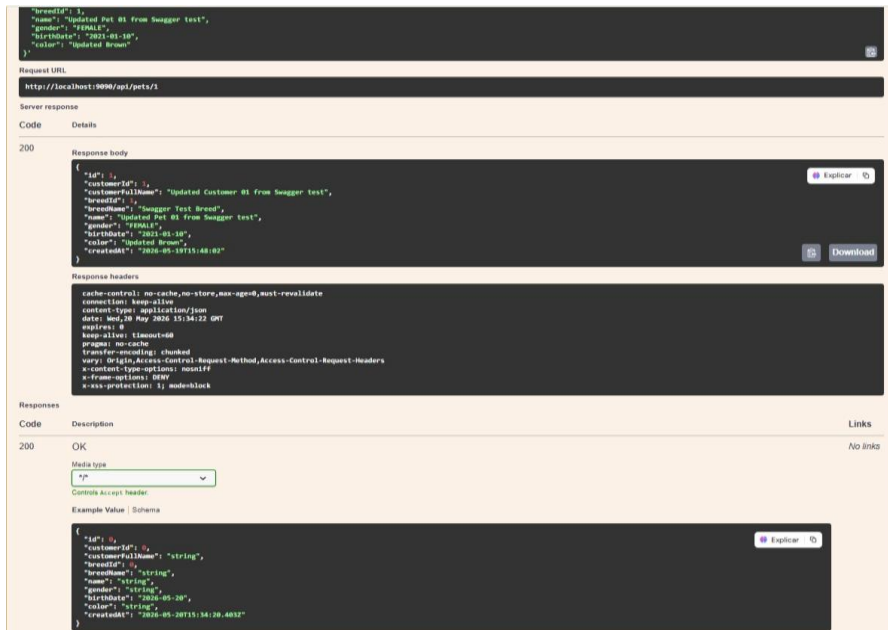
Responses:

curl -X 'POST' \
 http://localhost:3000/api/pets/1 \
 -H 'accept: */*' \
 -H 'Content-Type: application/json' \
 -d '{
 "id": 1,
 "customerId": 1,
 "breedId": 1,
 "name": "Updated Pet #1 from Swagger test",
 "gender": "FEMALE",
 "birthDate": "2021-01-10",
 "color": "Updated Brown"
 }'

Request URL: `http://localhost:3000/api/pets/1`

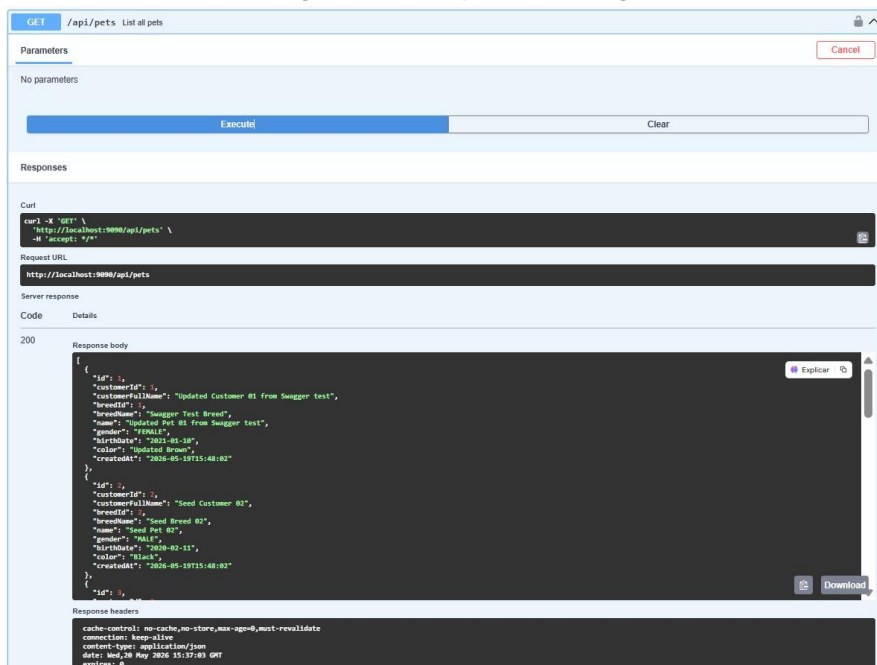
41. Confirmación de actualización de mascota

En esta imagen se evidencia la respuesta exitosa del sistema después de actualizar la información de la mascota. La API devuelve el registro modificado, confirmando que los cambios fueron almacenados correctamente y que se mantiene la relación con el cliente y la raza correspondiente.



42. Listado general de mascotas

Esta prueba valida la consulta general del módulo de mascotas. El sistema devuelve una lista estructurada con los registros existentes y sus datos principales. Esta operación permite visualizar de manera organizada los pacientes registrados en la clínica veterinaria.



43. Registro de nueva mascota

En esta prueba se valida la creación de una nueva mascota. Se envían datos como cliente propietario, raza, nombre, género, fecha de nacimiento y color. La respuesta exitosa confirma que el sistema permite registrar nuevos pacientes veterinarios de manera controlada.

POST

/api/pets Create a pet

Cancel

Reset

Parameters

No parameters

Request body required

application/json

```
{
  "customerId": 1,
  "breedId": 1,
  "name": "Swagger Test Pet",
  "photo": "swagger",
  "gender": "MALE",
  "birthDate": "2022-05-20",
  "color": "white"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:9090/api/pets' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "customerId": 1,
    "breedId": 1,
    "name": "Swagger Test Pet",
    "gender": "MALE",
    "birthDate": "2022-05-20",
    "color": "white"
  }'
```

Request URL

http://localhost:9090/api/pets

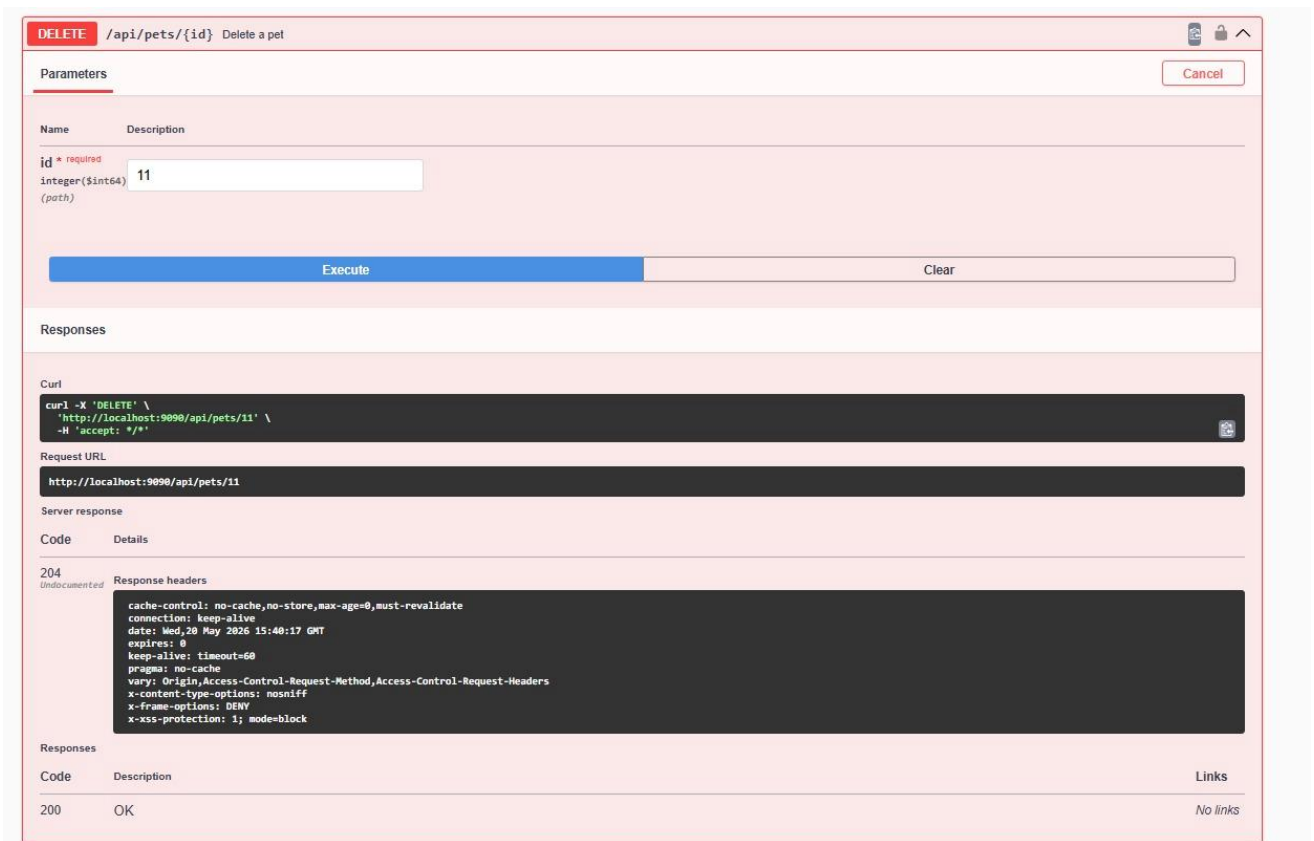
Server response

Code

Details

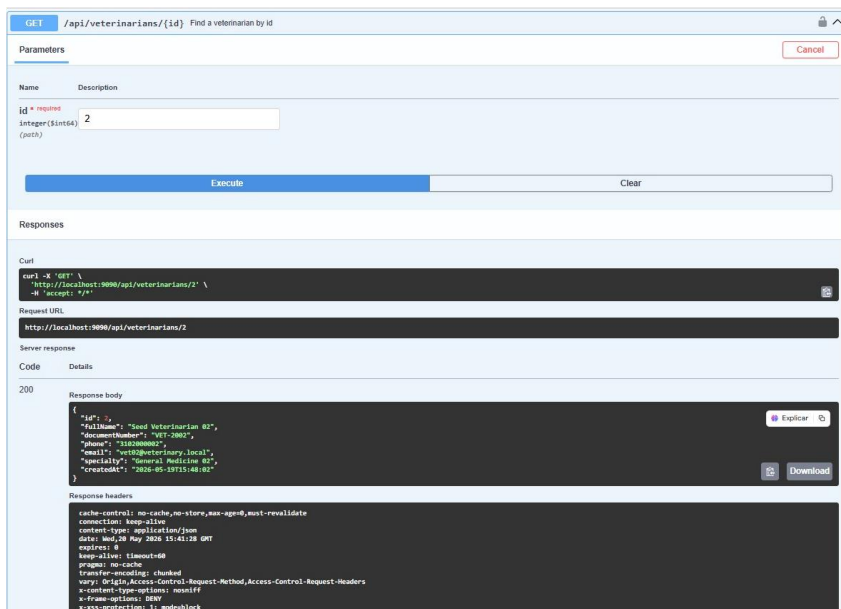
44. Eliminación de mascota

Esta prueba verifica la eliminación de una mascota registrada. El backend procesa la solicitud mediante el identificador enviado y confirma la operación. Esto demuestra que el módulo de mascotas permite gestionar adecuadamente el ciclo de vida de sus registros.³



45. Consulta individual de veterinario

En esta prueba se valida la consulta de un veterinario específico. El sistema responde con información profesional como nombre completo, documento, teléfono, correo electrónico y especialidad. Esta funcionalidad permite acceder de forma precisa a los datos del personal médico registrado.



46. Actualización de veterinario

Esta prueba verifica la actualización de la información de un veterinario existente. La API recibe datos personales y profesionales modificados, permitiendo mantener vigente la información del equipo médico. Esto resulta clave para la correcta administración del personal de la clínica.

The screenshot shows a REST client interface with the following sections:

- Parameters:** A table with columns 'Name' and 'Description'. It contains one parameter:

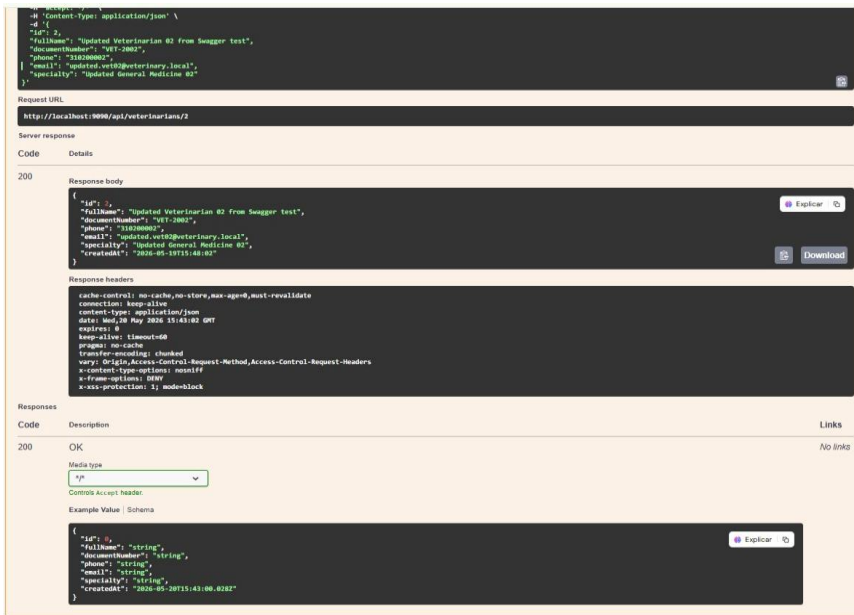
Name	Description
id * required integer(\$int64) (path)	2
- Request body:** A text area containing a JSON object:

```
{  "id": 2,  "fullName": "Updated Veterinarian 02 from Swagger test",  "documentNumber": "96172002",  "phone": "310200002",  "email": "updated.vet02@veterinary.local",  "specialty": "Updated General Medicine 02"}
```
- Buttons:** 'Execute' (blue) and 'Clear' (grey).
- Responses:** A section for the response, currently showing a curl command:

```
curl -X 'PUT' \  http://localhost:9090/api/veterinarians/2 \  -H 'accept: */*' \  -H 'Content-Type: application/json' \  -d '{  "id": 2,  "fullName": "Updated Veterinarian 02 from Swagger test",  "documentNumber": "96172002",  "phone": "310200002",  "email": "updated.vet02@veterinary.local",  "specialty": "Updated General Medicine 02"  }'
```
- Request URL:** `http://localhost:9090/api/veterinarians/2`

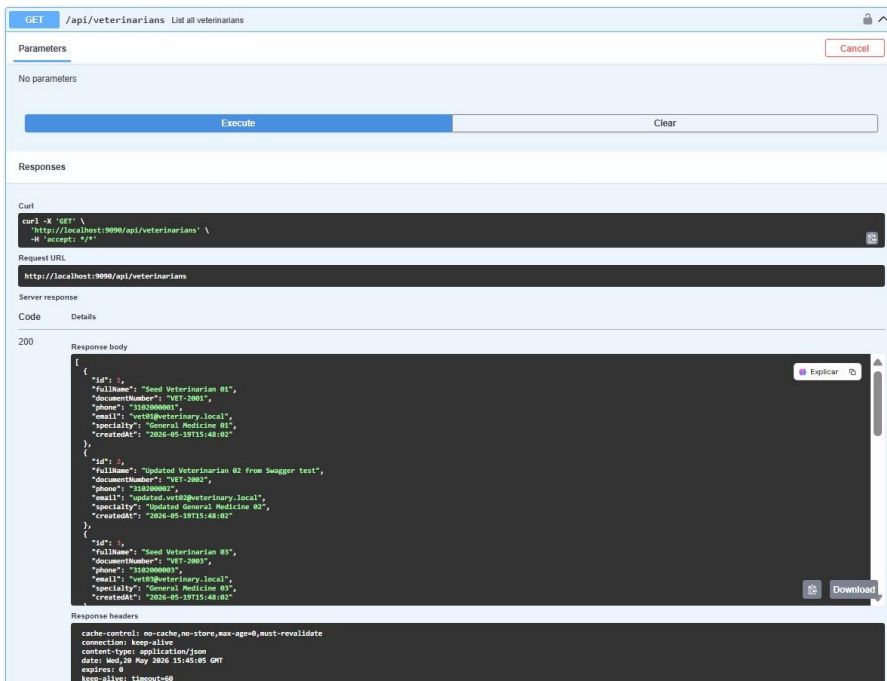
47. Confirmación de actualización de veterinario

En esta imagen se observa la respuesta exitosa después de modificar la información del veterinario. El sistema devuelve el registro actualizado, confirmando que los cambios fueron procesados correctamente y almacenados de forma consistente.



48. Listado general de veterinarios

Esta prueba valida la consulta general de los veterinarios registrados en el sistema. La API devuelve una lista organizada con la información principal de cada profesional. Esta funcionalidad facilita la gestión del talento médico disponible dentro de la clínica veterinaria.



49. Registro de nuevo veterinario

Esta prueba valida la creación de un nuevo veterinario dentro del sistema. Se ingresan datos como nombre completo, documento, teléfono, correo electrónico y especialidad. La

respuesta exitosa confirma que el backend permite registrar nuevos profesionales médicos de manera correcta y estructurada.

The screenshot shows a REST client interface with the following sections:

- Method and URL:** POST /api/veterinarians. A subtitle "Create a veterinarian" is present.
- Parameters:** A tab labeled "Parameters" with "No parameters" listed below it. "Cancel" and "Reset" buttons are on the right.
- Request body:** A tab labeled "Request body" with a "required" status and a dropdown menu set to "application/json". The body contains a JSON object:

```
{  "fullName": "Swagger Test Veterinarian",  "documentNumber": "VET-TEST-20260520",  "phone": "31699990001",  "email": "swagger.vet.test@veterinary.local",  "specialty": "Swagger Test Specialty"}
```
- Buttons:** "Execute" (blue) and "Clear" (grey) buttons are located below the request body.
- Responses:** A section titled "Responses" containing:
 - Curl:** A code block with the following command:

```
curl -X 'POST' \  'http://localhost:9090/api/veterinarians' \  -H 'accept: */*' \  -H 'Content-Type: application/json' \  -d '{  "fullName": "Swagger Test Veterinarian",  "documentNumber": "VET-TEST-20260520",  "phone": "31699990001",  "email": "swagger.vet.test@veterinary.local",  "specialty": "Swagger Test Specialty"  }'
```
 - Request URL:** A text field containing "http://localhost:9090/api/veterinarians".
 - Server response:** A section with "Code" and "Details" tabs, currently showing the "Code" tab.

50. Eliminación de veterinario

En esta prueba se valida la eliminación de un veterinario registrado. El sistema procesa la solicitud mediante el identificador correspondiente y confirma que el registro fue eliminado correctamente. Esto demuestra que el módulo de veterinarios cuenta con una gestión completa sobre sus registros.

DELETE /api/veterinarians/{id} Delete a veterinarian

Parameters

Name	Description
id * required	
integer(\$int64)	11
(path)	

Execute Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:9090/api/veterinarians/11' \
  -H 'accept: */*'
```

Request URL

http://localhost:9090/api/veterinarians/11

Server response

Code	Details
204	Response headers <pre>cache-control: no-cache,no-store,max-age=0,must-revalidate connection: keep-alive date: Wed, 20 May 2026 15:49:20 GMT expires: 0 keep-alive: timeout=60 pragma: no-cache vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers x-content-type-options: nosniff x-frame-options: DENY x-xss-protection: 1; mode=block</pre>

Responses

Code	Description	Links
200	OK	No links

Conclusión

Como resultado de las pruebas realizadas, se evidencia que el backend del Sistema de Gestión Veterinaria cumple con las operaciones principales requeridas para la administración de clientes, mascotas, veterinarios, citas, historiales médicos, tratamientos, especies y razas. La API demuestra una estructura funcional consistente, permitiendo gestionar información clínica y administrativa de manera organizada. Esto confirma que el sistema cuenta con una base sólida para integrarse posteriormente con una interfaz web o móvil y apoyar procesos reales dentro de una clínica veterinaria.